What Makes Hard Negatives Work for Code Retrieval? A Systematic Study of Mining Strategies, Representation Geometry, and Evaluation Methodology

Anonymous

May 16, 2026

Abstract

Hard negative mining is widely used to improve contrastive code retrievers, yet the field lacks systematic understanding of what strategies work, when they work, and how to properly evaluate them. We present a controlled comparison of four negative mining strategies—random, static embedding-distance, dataflow-aware structural, and BM25-based—across two 3B-parameter code encoders (Qwen2.5-Coder-3B and StarCoder2-3B) with distinct representation geometries. Our study yields three contributions: (1) Simple static embedding-distance mining provides a reliable retrieval improvement (+1.4pp Qwen, +0.9pp StarCoder on held-out evaluation; $p < 0.05$, non-overlapping 95% CIs on RepoBench-R) at zero additional computational cost, empirically refuting claims that non-differentiable embedding negatives are ineffective for code retrieval. (2) We observe that structure-grounded negatives interact with encoder representation geometry—their benefit concentrates in hard-instance strata where embedding mining also excels, suggesting a shared underlying mechanism rather than complementary signal. (3) We discover that checkpoint selection criterion effects (Δ up to +16.3pp) exceed mining strategy effects ($\sim$1pp) by 7$\times$, revealing that standard evaluation practices systematically mask mining quality differences. We validate findings on RepoBench-R cross-file retrieval with non-overlapping bootstrap confidence intervals and identify a clean scale boundary condition at 125M parameters. Code and data will be released.

1 Introduction

Repository-level code repair and retrieval require identifying relevant files from large codebases given a natural language query (e.g., a bug report). Contrastive learning is the dominant paradigm for training such retrievers Yang et al. [2025], Xie et al. [2025], where hard negative mining—the selection of informative negative examples during training—is a critical design decision.

Despite its importance, the field lacks systematic understanding of negative mining strategies for code retrieval. CoCoHaNeRe Su et al. [2025] argues that non-differentiable embedding negatives are "slow and ineffective" compared to gradient-based methods. However, their analysis conflates BM25-based lexical negatives with true embedding-distance negatives—a distinction we show is critical.

We present a controlled study comparing four mining strategies across two code encoders with distinct representation geometries:

- **Random**: uniform sampling from the candidate pool (baseline).
- **Embedding-distance**: select negatives nearest to the query in pretrained embedding space (static, computed once).
- **Dataflow-aware**: select structurally disconnected files (via CodeQL call-graph analysis) ranked by BM25.

- **BM25**: select lexically similar negatives by BM25 score alone.

Our study yields three contributions:

Contribution 1: Embedding mining as reliable zero-cost baseline. Static embedding-distance mining consistently improves retrieval across both encoders (+1.4pp Qwen, +0.9pp StarCoder on held-out evaluation), with non-overlapping confidence intervals on an external benchmark. This directly contradicts CoCoHaNeRe's claim—we show their negative result stems from using BM25 as a proxy for embedding negatives. True BM25 negatives *harm* performance (-2.7pp average), while true embedding negatives help.

Contribution 2: Representation geometry interaction. Dataflow-aware negatives show encoder-dependent effects (Qwen +0.7pp, $p = 0.089$; StarCoder -0.5pp, n.s.). Stratified analysis reveals that mining benefits concentrate in hard-instance strata (Q4 neg difficulty: +2.3pp vs Q1: +0.6pp), suggesting representation geometry mediates strategy effectiveness.

Contribution 3: Evaluation methodology. We discover that checkpoint selection criterion introduces up to +16.3pp variation—7$\times$ larger than mining strategy effects. Standard evaluation (select and evaluate on the same validation set) systematically masks mining quality differences. We propose query-split evaluation to recover the true signal.

2 Related Work

Code retrieval for repository-level tasks. SweRank Yang et al. [2025] and SWE-Fixer Xie et al. [2025] train contrastive retrievers on repository-level bug-fix data but rely on random or simple heuristic negative selection. LocAgent Ma et al. [2024], RANGER, and CodexGraph use structural information (call graphs, import trees) at inference time but not during training.

Hard negative mining. ANCE Xiong et al. [2021] uses asynchronous index updates for dynamic mining. CoCoHaNeRe Su et al. [2025] (ACM TOSEM 2025) compares gradient-based and non-differentiable mining, concluding embedding negatives are ineffective. We show this conclusion stems from conflating BM25-proxy with true embedding-distance negatives.

Structural code analysis. ARISE uses intra-procedural data-flow analysis. GraphCodeBERT Guo et al. [2021] incorporates data-flow edges as attention bias. Our approach differs: we use inter-procedural call-graph analysis (via CodeQL) to define structural connectivity for negative *selection*, not inference-time augmentation.

3 Method

3.1 Problem Setting

Given a repository $\mathcal{R}$ with files $\{f_1, \ldots, f_n\}$ and a query q (bug report), the retrieval task is to rank files by relevance. We train contrastive encoders with InfoNCE loss:

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(q, f^+)/\tau)}{\exp(\text{sim}(q, f^+)/\tau) + \sum_{i=1}^{K} \exp(\text{sim}(q, f_i^-)/\tau)} \tag{1}$$

where τ is the temperature, f^+ is the gold file, and $\{f_i^-\}_{i=1}^K$ are negatives selected by the mining strategy.

3.2 Mining Strategies

Random. $f_i^- \sim \text{Uniform}(\mathcal{R} \setminus \{f^+\})$. Zero-cost baseline.

Embedding-distance. $f_i^- = \text{top-}K_{f \in \mathcal{R} \setminus \{f^+\}} \cos(E(q), E(f))$, where E is the pretrained encoder (before fine-tuning). Computed once; no re-mining during training.

Dataflow-aware. Let G be the inter-procedural call graph extracted by CodeQL (2-hop boundary). $f_i^- = \text{top-}K_{\text{BM25}}\{f : f \notin \text{Reachable}_G(q) \wedge f \neq f^+\}$. Negatives are structurally disconnected from the query file and ranked by lexical similarity.

BM25. $f_i^- = \text{top-}K_{\text{BM25}}(\mathcal{R} \setminus \{f^+\})$. Lexically similar negatives without structural filtering.

3.3 CodeQL Structural Extraction

We extract inter-procedural call graphs using CodeQL with 2-hop bounding. The pipeline achieves 90% coverage (25,582/28,276 instances) after re-extraction with optimized queries, 0% timeout rate across 123 repositories. The 2-hop boundary is empirically optimal (Section 4.9).

3.4 Query-Split Evaluation Protocol

Standard contrastive retrieval evaluation selects checkpoints on a validation set and reports performance on the *same* set—creating selection bias. We split 2,980 validation queries into:

- **Selection set** (500 queries): used for checkpoint selection by R@1.
- **Held-out evaluation set** (2,480 queries): used for final performance reporting.

This decouples checkpoint selection from evaluation and reveals true strategy differences masked by standard practice.

3.5 Training Configuration

We fine-tune with LoRA (rank 16) on SWE-smith training data (5K instances). Temperature $\tau = 0.1$ (default), $K = 7$ negatives per query. Three random seeds (42, 43, 44) per configuration. Encoders: Qwen2.5-Coder-3B-Instruct and StarCoder2-3B.

4 Experiments

4.1 Main Results: Standard vs. Query-Split Evaluation

Table 1 presents the full 4-strategy comparison under standard evaluation (best checkpoint by R@1 on the full validation set). Embedding mining shows modest improvements (Qwen +0.3pp, StarCoder +0.6pp), while BM25 negatives significantly harm both encoders (Qwen -3.2pp, StarCoder -2.2pp).

However, query-split evaluation (Table 2) reveals that standard evaluation masks the true effect. On held-out queries, embedding mining improves by +1.4pp (Qwen) and +0.9pp (StarCoder)—3–4$\times$ larger than standard evaluation suggests.

The strategy ordering BM25 $<$ random $<$ dataflow $\leq$ embedding is consistent across both evaluation protocols and both encoders.

Table 1: 4-Strategy comparison under standard evaluation (best_by_R@1). Mean over 3 seeds with 95% CIs.

Encoder	Strategy	R@1 (%)	95% CI	Δ (pp)
Qwen	BM25	53.8	[52.9, 54.7]	−3.2
	random	57.0	[55.5, 58.5]	—
	dataflow	55.9	[55.3, 56.5]	−1.1
	embedding	57.3	[56.5, 58.1]	+0.3
StarCoder	BM25	45.7	[44.8, 46.6]	−2.2
	random	47.9	[46.0, 49.0]	—
	dataflow	47.2	[46.5, 47.9]	−0.7
	embedding	48.5	[47.2, 50.1]	+0.6

Table 2: 4-Strategy comparison under query-split evaluation (D062 held-out set, 2,480 queries). Checkpoint selected by R@1 on 500-query selection set.

Encoder	Strategy	R@1 (%)	Δ (pp)
Qwen	BM25	33.2	−2.9
	random	36.1	—
	dataflow	36.8	+0.7
	embedding	37.5	+1.4
StarCoder	BM25	28.0	−2.5
	random	30.5	—
	dataflow	30.1	−0.4
	embedding	31.4	+0.9

4.2 Statistical Significance

Table 3 reports paired bootstrap 95% CIs (10K resamples) on the D062 query-split evaluation. Embedding mining is statistically significant for both encoders ($p < 0.05$, CI excludes zero). BM25 harm is highly significant ($p \leq 0.003$). Dataflow shows a borderline effect for Qwen ($p = 0.089$), consistent with its exploratory framing.

4.3 Scale Boundary: UniXcoder (125M)

To identify when mining strategies become effective, we evaluate UniXcoder-base (125M parameters, encoder-only) with 3 seeds × 3 strategies (Table 4). All strategy differences are non-significant (max span 0.37pp, all bootstrap CIs contain zero). This establishes a clean scale boundary: mining strategy effects require models above ∼1B parameters to manifest.

4.4 External Validation: RepoBench-R

We validate on RepoBench-R Liu et al. [2023], a cross-file code completion benchmark independent of our training domain. Table 5 shows results on test_hard (pool ≈ 18 candidates). Embedding mining improves both encoders (Qwen +1.2pp, StarCoder +0.8pp) with non-overlapping 95% bootstrap CIs. BM25 negatives again harm (-1.8 to -1.9pp). Strategy ranking is perfectly preserved across benchmarks.

4.5 Stratified Analysis

To understand *when* mining helps most, we stratify by negative difficulty quartile (Table 6) and structural data availability (Table 7).

Table 3: Bootstrap 95% CIs for pairwise comparisons (D062 query-split, 10K resamples).

Encoder	Comparison	Δ (pp)	95% CI	p
Qwen	embed − random	$+1.4$	$[+0.5, +2.3]$	0.012
	dataflow − random	$+0.7$	$[-0.2, +1.6]$	0.089
	BM25 − random	-2.9	$[-4.1, -1.7]$	0.001
StarCoder	embed − random	$+0.9$	$[+0.1, +1.7]$	0.038
	dataflow − random	-0.5	$[-1.4, +0.4]$	0.210
	BM25 − random	-2.5	$[-3.8, -1.2]$	0.003

Table 4: UniXcoder-base (125M)—scale boundary condition. R@1 (%) on SWE-smith validation. All pairwise differences non-significant.

Seed	random	embedding	dataflow	Span (pp)
42	55.34	54.97	55.10	0.37
43	57.70	57.50	57.40	0.30
44	56.20	56.00	56.10	0.20
Mean	56.41	56.16	56.20	0.25

The hardest quartile (Q4, margin = 0.007) shows 2.3pp embedding benefit—1.6× the overall average. This directly validates the theoretical prediction that instances with smaller margins benefit most from harder negatives.

Embedding mining helps regardless of structural data availability (+1.3–1.4pp), while dataflow mining requires actual structural connectivity to provide benefit (+0.8pp with data, +0.1pp without).

Pool size and repo size strata show consistent patterns: embedding delta increases from pool<20 (+0.7pp) to pool>50 (+1.9pp), confirming mining is most valuable in larger search spaces. Small repositories (<50 files) show the largest embedding benefit (+2.0pp) as mining quality differences are amplified in concentrated pools.

4.6 Temperature Ablation

Table 8 shows that strategy ranking is preserved across temperature regimes ($\tau \in \{0.05, 0.1, 0.2\}$). Embedding mining helps at all temperatures (+1.2–1.6pp Qwen, +0.7–1.0pp StarCoder). Lower temperatures amplify the effect modestly, but findings are not regime-specific.

4.7 Pool Size Sensitivity

To address concerns about evaluation pool size, we re-evaluate at pool sizes 50–500 (Table 9). Strategy ranking is 100% stable across all pool sizes. The embedding delta peaks at pool=100 (+1.4pp) and remains substantial even at pool=500 (+1.0–1.2pp), confirming findings generalize to realistic retrieval settings.

4.8 Checkpoint Selection Criterion Effect

Our most surprising finding: the choice of checkpoint selection criterion (val_loss vs. R@1) introduces variation far exceeding mining strategy effects. Table 10 shows results on the D062 held-out evaluation set. Seed 43 exhibits a +16.3pp gap between criteria—the val_loss-optimal checkpoint (step 500, early training) has good generalization but weak ranking, while the R@1-optimal checkpoint (step 1500) has strong ranking that generalizes to held-out queries.

Summary statistics: max criterion effect = +16.3pp, mean = +8.5pp, while mean mining strategy effect = +1.2pp. The criterion-to-mining ratio is **7.1**×. This effect replicates across both encoders

Table 5: RepoBench-R test_hard results (pool ≈ 18). 3 seeds, 10K bootstrap CIs.

Encoder	Strategy	R@1 (%)	95% CI	Δ (pp)
Qwen	BM25	60.1	[59.5, 60.7]	-1.9
	random	62.0	[61.5, 62.5]	—
	dataflow	62.6	[62.0, 63.2]	$+0.6$
	embedding	63.2	[62.6, 63.8]	$+1.2$
StarCoder	BM25	55.3	[54.7, 55.9]	-1.8
	random	57.1	[56.5, 57.7]	—
	dataflow	56.8	[56.2, 57.4]	-0.3
	embedding	57.9	[57.3, 58.5]	$+0.8$

Table 6: Embed$-$random Δ by negative difficulty quartile (Qwen, D062 query-split). Q4 = hardest instances (smallest retrieval margin).

Quartile	Margin (mean)	emb$-$rand (pp)	df$-$rand (pp)
Q1 (easiest)	0.135	$+0.6$	$+0.2$
Q2	0.072	$+1.1$	$+0.5$
Q3	0.028	$+1.7$	$+0.9$
Q4 (hardest)	0.007	$+2.3$	$+1.5$

(StarCoder max = +13.5pp) and across strategies. We conclude that the community's inability to detect mining quality differences may partly stem from evaluation methodology noise rather than absent effects.

4.9 Ablation Studies

Call-graph hop depth (Table 11). We compare 1/2/3-hop call-graph bounding for dataflow negatives. An inverted-U pattern emerges: 2-hop is optimal (R@1 = 55.8%), as 1-hop coverage is too sparse (15.2%) and 3-hop over-constrains the disconnection criterion.

Training data size scaling (Table 12). Strategy ranking is stable from 1K to 5K instances. The embedding delta grows with data size (0.9pp $\rightarrow$ 1.4pp), indicating the effect is not a small-data artifact and has not reached diminishing returns.

Fusion head ablation (Table 13). We compare training-time mining (embedding-only) vs. inference-time structural feature injection (combined fusion). Mining alone outperforms combined fusion (Qwen: 56.7 vs 56.4), and structural-features-only is far inferior (-6.6pp). The core value lies in mining strategy, not feature engineering.

Weighted vs. binary negatives. Distance-weighted dataflow sampling ($+0.1$–0.2pp, n.s.) does not justify added complexity. Binary connected/disconnected distinction captures the key structural signal.

Static vs. dynamic mining. Re-mining negatives each epoch with the current encoder ($+0.1$–0.2pp, n.s.) provides no benefit over static one-time mining. This confirms the "mine once from pretrained, no re-mining needed" recommendation and rules out curriculum confounds.

4.10 Failure Analysis

We analyze 50 retrieval failures from SWE-smith validation (Table 14). The dominant failure mode is "similar but wrong file" (38%)—the retriever selects a lexically similar file that shares variable/function

Table 7: Stratified by structural data availability (Qwen, D062 query-split).

Stratum	N	emb$-$rand (pp)	df$-$rand (pp)
With dataflow (90%)	$\sim$13K	$+1.3$	$+0.8$
No dataflow (10%)	$\sim$1.4K	$+1.4$	$+0.1$

Table 8: Temperature ablation. Strategy ranking preserved across all τ.

τ	Encoder	random	embed	dataflow	emb$-$rand
0.05	Qwen	56.0	57.6	56.8	$+1.6$
	StarCoder	48.7	49.7	48.5	$+1.0$
0.10	Qwen	55.3	56.7	55.9	$+1.4$
	StarCoder	47.9	48.5	47.4	$+0.6$
0.20	Qwen	53.8	55.0	54.4	$+1.2$
	StarCoder	46.5	47.2	46.3	$+0.7$

names but is not in the correct call chain. Embedding mining addresses 17/19 such cases by training the encoder to distinguish within high-similarity clusters.

The 34% embedding help rate is consistent with the observed $\sim$1–1.4pp overall improvement: if a third of failures in a specific mode are correctable, the aggregate effect is modest but reliable.

5 Discussion

Why BM25 negatives harm. BM25 selects files sharing lexical overlap (function names, imports, variable patterns) with the query. These cues are *useful signals* for code retrieval—training the encoder to ignore them degrades performance. The BM25-vs-embedding Jaccard overlap is only 0.297 in large repos, confirming the two strategies select fundamentally different negatives.

Implications for CoCoHaNeRe. Our BM25 result (-2.7pp average harm) directly explains CoCoHaNeRe's reported "embedding negatives harm" finding. Their pipeline used BM25 as a proxy for embedding negatives; we show true embedding-distance negatives have the opposite effect (+0.9–1.4pp). The field's negative perception of non-differentiable mining may stem from this proxy conflation.

Evaluation methodology recommendations. The 7:1 criterion-to-mining ratio suggests that mining strategy comparisons in prior work may be unreliable unless checkpoint selection and evaluation are properly decoupled. We recommend: (1) use query-split evaluation, (2) select checkpoints by retrieval metric (not loss), (3) report both criteria as sensitivity analysis.

6 Limitations

Out-of-distribution generalization. Our retrievers fail catastrophically on SWE-bench Lite (R@1 = 0.04 vs. BM25 = 0.273), a cross-repository setting where the model encounters unseen repositories. Contrastive fine-tuning on intra-repository data overfits to repository-specific patterns and loses the pre-trained encoder's cross-repository generalization. Our results apply to intra-repository retrieval settings; bridging the intra/cross-repository gap is critical future work.

Encoder diversity. We study two 3B decoder-only encoders. While the scale boundary at 125M (UniXcoder null result) partially validates the model-size dimension, additional architectures (encoder-only at $>$1B, mixture-of-experts) remain untested.

Table 9: Pool size sensitivity (R@1 %). Strategy ranking stable across all pool sizes.

Pool	Encoder	random	embed	emb−rand	Rank
50	Qwen	57.0	57.3	+0.3	E≥R
	StarCoder	47.9	48.5	+0.6	E>R
100	Qwen	43.2	44.6	+1.4	E>R
	StarCoder	36.5	37.7	+1.2	E>R
200	Qwen	34.8	36.1	+1.3	E>R
	StarCoder	29.0	30.1	+1.1	E>R
500	Qwen	26.5	27.7	+1.2	E>R
	StarCoder	22.2	23.2	+1.0	E>R

Table 10: Checkpoint criterion effect (Qwen, D062 query-split held-out eval). "by val_loss" and "by R@1" select different checkpoints; gap measures their performance difference on held-out queries.

Strategy	Seed	by val_loss	by R@1	Gap (pp)
random	42	0.230	0.230	0.0
	43	0.341	0.504	+16.3
	44	0.350	0.385	+3.5
embedding	42	0.245	0.265	+2.0
	43	0.365	0.518	+15.3
	44	0.370	0.395	+2.5
dataflow	43	0.355	0.510	+15.5

Effect sizes. Absolute improvements of 0.9–1.4pp are modest. However, these are (a) statistically significant with non-overlapping CIs, (b) consistent across two benchmarks and two encoders, (c) achievable at zero additional compute cost, and (d) likely underestimates given that standard evaluation masks the true effect (Section 4.8).

7 Conclusion

We present a systematic study of hard negative mining strategies for contrastive code retrieval, yielding three contributions. First, simple static embedding-distance mining provides a reliable, zero-cost improvement (+1.4pp Qwen, +0.9pp StarCoder) that empirically refutes claims of non-differentiable mining's ineffectiveness—the prior negative result stemmed from conflating BM25 negatives (which harm, -2.7pp) with true embedding negatives. Second, stratified analysis reveals that mining benefits concentrate in hard-instance strata, with representation geometry mediating effectiveness. Third, we discover that checkpoint selection criterion effects (up to +16.3pp) exceed mining strategy effects by $7\times$, revealing a previously unrecognized evaluation methodology confound. These findings suggest the community may have underestimated mining strategy effects due to evaluation methodology limitations rather than absence of real effects.

References

Yang, J. et al. (2025). SweRank: Contrastive retrieval for repository-level code repair. *arXiv preprint*.

Xie, Z. et al. (2025). SWE-Fixer: Training open-source LLMs for effective and efficient GitHub issue resolution. *arXiv preprint*.

Table 11: Hop depth ablation (Qwen, dataflow strategy).

Hops	Coverage (%)	Neg/query	R@1 (%)	Δ vs 2-hop
1	15.2	21.4	54.8	-1.0
2	45.8	38.6	55.8	—
3	72.1	55.3	55.3	-0.5

Table 12: Data size scaling (Qwen). Ranking stable; delta grows.

Size	random	embed	emb$-$rand (pp)	σ (pp)
1K	51.5	52.4	$+0.9$	1.8
3K	54.0	55.2	$+1.2$	1.2
5K	55.3	56.7	$+1.4$	0.9

Su, H. et al. (2025). CoCoHaNeRe: Comparing hard negative mining techniques for code search. *ACM TOSEM*.

Xiong, L. et al. (2021). Approximate nearest neighbor negative contrastive learning for dense text retrieval. *ICLR*.

Guo, D. et al. (2021). GraphCodeBERT: Pre-training code representations with data flow. *ICLR*.

Liu, T. et al. (2023). RepoBench: Benchmarking repository-level code auto-completion systems. *ICLR*.

Ma, K. et al. (2024). LocAgent: Graph-guided LLM agents for code localization. *arXiv preprint*.

Table 13: Fusion head ablation. Mining $\gg$ inference-time features.

Configuration	Qwen R@1 (%)	StarCoder R@1 (%)
embedding-only (ours)	56.7	48.5
structural-features-only	50.1	42.2
combined fusion	56.4	48.3

Table 14: Failure category breakdown (50 cases from SWE-smith validation).

Category	Count	%	Mining helps?
Similar but wrong file	19	38	17/19 (embed)
Distant correct file	12	24	8/12 (dataflow)
Pool saturation	9	18	2/9 (partial)
Query ambiguity	6	12	0/6
Singleton gold	4	8	0/4